

```

    T.print(T.root, '\n');
    return 0;
}
/*
注意：这里使用了一种不带旋转且支持分裂合并的 Treap 替代 Splay
同类题目：Jewel Magic, UVa 11996
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表4-5所示。

表4-5 排序二叉树例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-24	UVa 11020	Efficient Solutions	map 维护点集；单调性	刘汝佳
例 4-25	HDU 3726	Graph and Queries	名次树；并查集；时光倒流	刘汝佳
例 4-26	UVa 11922	Permutation Transformer	伸展树/splay；Treap；可分裂合并的序列	刘汝佳、魏子豪、陈锋

4.4 树的经典问题与方法

与树相关的问题在竞赛中常考，尤其是在国内的全国青少年信息学奥林匹克竞赛中。本节提供了一些与树相关的经典例题及其代码实现。

例4-27 【倍增LCA】村庄有多远（How far away, HDU 2586）

一个村庄有 n ($2 \leq n \leq 40\,000$) 栋房子和 $n-1$ 条双向路，每两栋房子之间都有一条唯一路径，任意一条路径都给出一个长度。现在有 m 次询问，每个询问给出 u, v ，求 u, v 之间的距离。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
const int MAXN = 40000 + 4;
int N, L, Tin[MAXN], Tout[MAXN], UP[MAXN][18], timer;
struct Edge {
    int v, k;
    Edge(int _v, int _k) : v(_v), k(_k) {}
};
vector<Edge> G[MAXN];
int Dist[MAXN], D[MAXN]; // 到 root 的距离, 深度
void dfs(int u, int fa) { // LCA 预处理
    Tin[u] = ++timer, UP[u][0] = fa;
    if (u) D[u] = D[fa] + 1;
    for (int i = 1; i < L; i++) UP[u][i] = UP[UP[u][i - 1]][i - 1];
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge& e = G[u][i];
        if (e.v != fa) Dist[e.v] = Dist[u] + e.k, dfs(e.v, u);
    }
    Tout[u] = ++timer;
}
bool isAncestor(int u, int v) { return Tin[u] <= Tin[v] && Tout[u] >= Tout[v]; }
int LCA(int u, int v) {
    if (D[u] > D[v]) return LCA(v, u); // 保证 u 的深度小于 v 的深度
    if (isAncestor(u, v)) return u; // u 是 v 的祖先
    for (int i = L; i >= 0; --i)
        if (!isAncestor(UP[u][i], v)) u = UP[u][i];
    return UP[u][0];
}
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T, M, u, v, k;
    L = ceil(log2(N));
    cin >> T;
    while (T--) {
        cin >> N >> M;
        for (int i = 0; i < N; i++) G[i].clear();
        for (int i = 0; i < M - 1; i++) {
            cin >> u >> v >> k, u--, v--;
            G[u].push_back(Edge(v, k)), G[v].push_back(Edge(u, k));
        }
        memset(UP, 0, sizeof(UP));
        Dist[0] = 0, D[0] = 0;
        dfs(0, 0);
        for (int i = 0; i < M; i++) {
            cin >> u >> v, u--, v--;
            cout << Dist[u] + Dist[v] - 2 * Dist[LCA(u, v)] << endl;
        }
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.7 节例题 33
注意: 本题充分利用 DFS 时间戳, 简化了祖先关系的判断操作, 并且将查找 LCA 的过程简化到只有一个
循环
同类题目: Rikka with Intersection of Paths, Codeforces Gym 102012G
POJ 3417, Network
*/

```

例4-28 【树的点分治；路径统计】路径统计 (Tree, POJ 1741)

给定一棵 n 个结点的正权树，定义 $\text{dist}(u, v)$ 为 u, v 两点间唯一路径的长度（即所有边的权值和），再给定一个正数 K ，统计有多少对结点 (a, b) 满足 $\text{dist}(a, b) \leq K$ 。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <cassert>
#include <vector>
#include <algorithm>
#include <iterator>

using namespace std;
const int INF = 2147483647, MAXN = 10000 + 4;

struct Edge {
    int v, w;
    Edge(int _v, int _w): v(_v), w(_w) {}
};
int N, K;
vector<Edge> G[MAXN];
bool VIS[MAXN];

int get_size(int u, int fa) {          // 子树 u 的体积
    assert(!VIS[u]);
    int ans = 1;
    for (size_t i = 0; i < G[u].size(); i++) {
        int v = G[u][i].v;
        if (v == fa || VIS[v]) continue;
        ans += get_size(v, u);
    }
    return ans;
}

// 给出子树 u 的大小, 找出其重心
int find_centroid(int u, int fa, int usz, int &ch_sz, int &ct) {
    assert(!VIS[u]);
    int sz = 1, max_ch = -INF;
    for (size_t i = 0; i < G[u].size(); i++) {
        int v = G[u][i].v;
        if (v == fa || VIS[v]) continue;
        int chsz = find_centroid(v, u, usz, ch_sz, ct);
        sz += chsz, max_ch = max(max_ch, chsz);
    }
    max_ch = max(max_ch, usz - sz);
    if (max_ch < ch_sz) ch_sz = max_ch, ct = u;
    return sz;
}

int find_centroid(int u) {            // 子树 u 的重心
    int ch_sz = INF, ct = -1, sz = get_size(u, -1);
    find_centroid(u, -1, sz, ch_sz, ct);
    assert(ct != -1 && ch_sz <= sz / 2);
}

```

```

    return ct;
}

// 收集子树 u 中所有到 u 的小于等于 K 的路径长度
void get_paths(int u, int fa, int plen, vector<int>& paths) {
    if (plen > K) return;
    paths.push_back(plen);
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge &e = G[u][i];
        if (e.v != fa && !VIS[e.v])
            get_paths(e.v, u, plen + e.w, paths);
    }
}

// 统计 P 中两个元素之和小于等于 K 的 pair 个数
inline int count_pairs(vector<int>& P) {
    sort(P.begin(), P.end());
    int ans = 0;
    for (int l = 0, r = P.size() - 1; ; l++) {
        while (r > l && P[r] + P[l] > K) r--;
        if (r <= l) break;
        ans += r - l;
    }
    return ans;
}

int solve(int u) {
    int ans = 0;
    vector<int> lens;
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge &e = G[u][i];
        if (VIS[e.v]) continue;
        vector<int> ps;
        get_paths(e.v, u, e.w, ps);
        copy(ps.begin(), ps.end(), back_inserter(lens));
    }
    ans += count_pairs(lens) + lens.size();
    VIS[u] = true;
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge &e = G[u][i];
        if (!VIS[e.v]) ans += solve(find_centroid(e.v));
    }
    return ans;
}

int main() {
    while (scanf("%d%d", &N, &K) == 2 && (N || K)) {
        for (int i = 0; i <= N; i++) G[i].clear(), VIS[i] = false;
        for (int i = 0, u, v, w; i < N - 1; i++) {

```

```

scanf("%d%d%d", &u, &v, &w), u--, v--;
G[u].push_back(Edge(v, w)), G[v].push_back(Edge(u, w));
}
printf("%d\n", solve(find_centroid(0)));
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.7 节例题 35
*/

```

例4-29 【树的点分治；路径统计】铁人比赛（Ironman Race in Treeland, ACM/ICPC Kuala Lumpur 2008, UVa 12161）

给定一棵 n ($1 \leq n \leq 30\,000$) 个结点的树，每条边包含长度 L 和费用 D ($1 \leq D, L \leq 1000$) 两个权值。要求选择一条总费用不超过 m ($1 \leq m \leq 10^8$) 的路径，使得路径总长度尽量大。输入应保证有解。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
struct Edge { int v, d, l; };
const int INF = 0x3f3f3f3f, MAXN = 3e4 + 10;
typedef map<int, int>::iterator MIT;

int N, M, MaxSub[MAXN], SZ[MAXN], VIS[MAXN], Dep[MAXN], Cost[MAXN];
// MaxSub[i]: 去除结点 i 后得到的森林中结点数最多的树的结点数; SZ[u]: 子树 u 的体积;
// Dep: 长度; Cost: 路径的费用
vector<Edge> G[MAXN];
void find_center(int u, int fa, const int tree_sz, int& center) { // 找重心
    int &szu = SZ[u], &msu = MaxSub[u];
    szu = 1, msu = 0;
    for (const Edge& e : G[u]) {
        if (e.v == fa || VIS[e.v]) continue;
        find_center(e.v, u, tree_sz, center);
        szu += SZ[e.v], msu = max(msu, SZ[e.v]);
    }
    msu = max(msu, tree_sz - SZ[u]);
    if (MaxSub[center] > msu) center = u;
}

void insert_cd(map<int, int>& ps, int c, int d) {
    if (c > M) return;
    MIT it = ps.upper_bound(c);
    if (it == ps.begin() || (--it)->second < d) { // 保证 ps 里面{费用:长度}同时递增
        ps[c] = d; // (it-1)->c ≤ c, 要求 d > (it-1)->d 才插入 c:d
        it = ps.upper_bound(c); // 对于所有的 it(it->c > c), 要求 it->d > d, 否则删除
    }
}

```

```

        while (it != ps.end() && it->second <= d) ps.erase(it++);
    }
}

void collect_deps(int u, int fa, map<int, int>& ps) { // 子树 u 结点路径的费用:长度
    SZ[u] = 1;
    insert_cd(ps, Cost[u], Dep[u]);
    for (const Edge& e : G[u]) {
        if (e.v == fa || VIS[e.v]) continue;
        Dep[e.v] = Dep[u] + e.l, Cost[e.v] = Cost[u] + e.d;
        collect_deps(e.v, u, ps), SZ[u] += SZ[e.v];
    }
}

void count(int u, int& max_len) { // 计算经过子树 u 根结点的路径数
    map<int, int> ps, vps; // u 子树, v 子树中的费用:长度
    ps[0] = 0;
    for (const Edge& e : G[u]) {
        if (VIS[e.v]) continue;
        Dep[e.v] = e.l, Cost[e.v] = e.d;
        vps.clear(), collect_deps(e.v, u, vps);
        for (const pair<int, int>& p : vps) {
            MIT it = ps.upper_bound(M - p.first);
            if (it != ps.begin()) max_len = max(max_len, p.second + (--it)->second);
        }
        for (const pair<int, int>& p : vps) insert_cd(ps, p.first, p.second);
    }
}

void solve(int u, int& max_len) {
    count(u, max_len), VIS[u] = true;
    for (const Edge& e : G[u]) {
        if (VIS[e.v]) continue;
        int center = 0;
        find_center(e.v, u, SZ[e.v], center), solve(center, max_len);
    }
}

int main() {
    int T;
    scanf("%d", &T);
    for (int kase = 1; kase <= T; kase++) {
        scanf("%d%d", &N, &M);
        fill_n(VIS, N + 1, 0), MaxSub[0] = N;
        for (int i = 1; i <= N; i++) G[i].clear();
        for (int i = 1, u, v, d, l; i < N; i++) {
            scanf("%d%d%d%d", &u, &v, &d, &l);
            G[u].push_back({v, d, l}), G[v].push_back({u, d, l});
        }
    }
}

```



```

    int center = 0, max_len = 0;
    find_center(1, -1, N, center);          // 找到初始的重心
    solve(center, max_len);                 // 递归求解
    printf("Case %d: %d\n", kase, max_len);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.7 节例题 36
注意：本题中如何使用 map 来维护一个单调序列
同类题目：Race, IOI 2011, 牛客 NC 51143
*/

```

例 4-30 【树链剖分；维护边权；树上差分】闪电的能量 (Lightning Energy Report, ACM/ICPC Jakarta 2010, UVa 1674)

有 n ($n \leq 50\,000$) 座房子形成树状结构，还有 Q ($Q \leq 10\,000$) 道闪电。闪电每次都会打到两座房子 a, b 上，你需要把二者路径上所有点（包括 a, b ）的闪电值加上 c ($c \leq 100$)，最后输出每座房子的总闪电值。

【代码实现（基于树链剖分）】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)
typedef long long LL;

const int MAXN = 65536;
struct SegTree {
    int addv[MAXN * 4], N;
    void update(int o, int L, int R, int qL, int qR, int val) {
        if (qL <= L && R <= qR) {
            addv[o] += val; // 区间加上 val
            return;
        }
        int M = (L + R) / 2;
        if (qL <= M) update(o << 1, L, M, qL, qR, val); // 覆盖左区间
        if (M < qR) update(o << 1 | 1, M + 1, R, qL, qR, val); // 覆盖右区间
    }

    void add(int qL, int qR, int val) { update(1, 1, N, qL, qR, val); }

    void init(int o, int L, int R) { // 初始化线段树
        addv[o] = 0;
        if (L == R) return;
        int M = (L + R) / 2;
        init(o << 1, L, M), init(o << 1 | 1, M + 1, R);
    }
}

```

```

int query(int o, int L, int R, int qv, int val) {
    if (L == R) return val + addv[o]; // 找到答案并且将答案返回
    int M = (L + R) >> 1;
    if (qv <= M) return query(o<<1, L, M, qv, val + addv[o]); // 答案在左区间
    return query(o<<1|1, M + 1, R, qv, val + addv[o]); // 答案在右区间
}

// Fa[i]: i 的父结点; HcHead[i]: i 所在重链头; HSon[i]: i 重儿子;
// SZ[i]: 子树体积; ID[i]: i 在线段树中序号
int Fa[MAXN], HcHead[MAXN], Depth[MAXN], HSon[MAXN], SZ[MAXN], ID[MAXN], intSz;

SegTree ST;
vector<int> G[MAXN]; // 存储图
int dfs(int u, int fa) { // 第一次 DFS, 得到每个结点的重儿子、深度和父结点
    SZ[u] = 1, Fa[u] = fa, HSon[u] = 0, Depth[u] = Depth[fa] + 1;
    for (auto v : G[u]) {
        if (v == fa) continue;
        SZ[u] += dfs(v, u);
        if (SZ[v] > SZ[HSon[u]]) HSon[u] = v; // 重儿子为体积最大的子树
    }
    return SZ[u];
}

void hld(int u, int fa, int x) { // 得到结点在线段树中的标号及重链的标号
    ID[u] = ++intSz, HcHead[u] = x; // 重链的标号为该重链最顶端的结点
    // 先处理重链, 保证剖分完后每条重链中的标号是连续的
    if (HSon[u])
        hld(HSon[u], u, x);
    for (auto v : G[u])
        if (v != fa && v != HSon[u]) hld(v, u, v);
}

void addPath(int u, int v, int w) {
    while (true) {
        int hu = HcHead[u], hv = HcHead[v];
        if (hu == hv) break; // 直到两点位于同一条重链才停止
        if (Depth[hu] < Depth[hv]) swap(u, v), swap(hu, hv); // 更新 h→head()
        ST.add(ID[hu], ID[u], w), u = Fa[hu];
    }
    if (Depth[u] < Depth[v]) swap(u, v);
    ST.add(ID[v], ID[u], w); // 更新 u→v
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (int kase = 1, Q, N; kase <= T; kase++) {
        cin >> N;
        assert(N < MAXN);
    }
}

```

```

ST.N = N, ST.init(1, 1, N);
for (int i = 1; i <= N; i++) G[i].clear();
SZ[0] = 0, Depth[1] = 0;
for (int i = 1, u, v; i < N; i++)
    cin >> u >> v, G[u + 1].push_back(v + 1), G[v + 1].push_back(u + 1);
dfs(1, 1);
intSz = 0;
hld(1, 1, 1);
cin >> Q;
for (int i = 0, u, v, w; i < Q; i++)
    cin >> u >> v >> w, addPath(u + 1, v + 1, w);
printf("Case #%d:\n", kase);
for (int i = 1; i <= N; i++) printf("%d\n", ST.query(1, 1, N, ID[i], 0));
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.7 节例题 38
*/

```

【代码实现（基于LCA与树上差分）】

```

// 基于 LCA 的树上差分解法, 陈锋
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 50000 + 4;

template <int SZ>
struct LCA {
    vector<int> G[MAXN];
    int N, L, Tin[MAXN], Tout[MAXN], UP[MAXN][18], timer; // LCA 相关
    void init(int _n) {
        N = _n, L = ceil(log2(N)), timer = 0;
        for (int i = 0; i <= N; i++) G[i].clear();
    }
    void addEdge(int u, int v) { G[u].push_back(v), G[v].push_back(u); }
    void dfs(int u, int fa = 0) {
        Tin[u] = ++timer, UP[u][0] = fa;
        for (int i = 1; i <= L; i++) UP[u][i] = UP[UP[u][i - 1]][i - 1];
        for (size_t i = 0; i < G[u].size(); i++)
            if (G[u][i] != fa) dfs(G[u][i], u);
        Tout[u] = ++timer;
    }

    bool isAncestor(int u, int v) { return Tin[u] < Tin[v] && Tout[v] < Tout[u]; }

    int lca(int u, int v) {
        if (u == v) return u;
        if (isAncestor(u, v)) return u;
        if (isAncestor(v, u)) return v;
    }
};

```

```

        for (int i = L; i >= 0; --i)
            if (!isAncestor(UP[u][i], v)) u = UP[u][i];
        return UP[u][0];
    }
};

LCA<MAXN> lca;
int mark[MAXN], ans[MAXN];

int dfs_mark(int u, int fa) {
    int &a = ans[u];
    a = mark[u];
    for (size_t i = 0; i < lca.G[u].size(); i++) {
        int v = lca.G[u][i];
        if (v == fa) continue;
        a += dfs_mark(v, u);
    }
    return a;
}

int main() {
    int T;
    cin >> T;
    for (int kase = 1, N, Q, x, y; kase <= T; kase++) {
        cin >> N, lca.init(N);
        for (int i = 1; i < N; i++) cin >> x >> y, lca.addEdge(x, y);
        lca.dfs(0);
        cin >> Q;
        fill_n(mark, N + 1, 0);
        for (int i = 0, c; i < Q; i++) {
            cin >> x >> y >> c;
            int d = lca.lca(x, y), pd = lca.UP[d][0];
            mark[x] += c, mark[y] += c, mark[d] -= c;
            if (pd != d) mark[pd] -= c;
        }
        dfs_mark(0, 0);
        printf("Case #%d:\n", kase);
        for (int i = 0; i < N; i++) printf("%d\n", ans[i]);
    }
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.7 节例题 38
注意：本题中树上差分的实现要注意 lca 的父亲判断
*/

```

例4-31 【树链剖分；维护点权；子树点权】软件包管理器（NOI 2015，牛客NC 17882）

Linux用户和OSX用户一定对软件包管理器不会陌生。使用软件包管理器，用户可以通过一行命令快速安装某个软件包。而从软件源下载软件时，需要解决所有依赖（即还需下载安装这个软件包所依赖的其他软件包）。所有的配置工作均可由软件包管理器自动完成。Debian/Ubuntu使用的apt-get、Fedora/CentOS使用的yum以及OSX下可用的homebrew都是优秀的软件包管理器。

假设你决定设计自己的软件包管理器，就需要解决软件包之间的依赖问题。如果软件包A依赖于软件包B，那么安装软件包A以前必须先安装软件包B。同时，如果想要卸载软件包B，则必须先卸载软件包A。现在你已经获得了所有软件包之间的依赖关系。除0号软件包外，管理器中的软件包都会依赖一个且仅一个软件包，而0号软件包不依赖任何一个软件包。不存在循环依赖，当然也不存在某个软件包依赖自己。

现在你要为你的软件包管理器写一个依赖解决程序。根据反馈，用户希望在安装和卸载某个软件包时，快速地知道这个操作实际上会改变多少个软件包的安装状态（即安装操作会安装多少个未安装的软件包，或卸载操作会卸载多少个已安装的软件包），你的任务就是实现这个功能。注意，安装一个已安装的软件包或卸载一个未安装的软件包，都不会改变任何软件包的安装状态，即在此情况下，改变安装状态的软件包数为0。

输入第一行包含一个整数 n ($n \leq 10^5$)，表示软件包的总数。软件包从0开始编号。随后一行包含 $n-1$ 个整数，相邻整数之间用单个空格隔开，分别表示 $1, 2, 3, \dots, n-2, n-1$ 号软件包依赖的软件包的编号。接下来的一行包含一个整数 q ($q \leq 10^5$)，表示询问的总数。之后 q 行，每行一个询问。询问分为以下两种。

- ☐ install x : 表示安装软件包 x 。
- ☐ uninstall x : 表示卸载软件包 x 。

你需要维护每个软件包的安装状态，一开始所有的软件包都处于未安装状态。对于每个操作，你需要输出这步操作会改变多少个软件包的安装状态，随后使用这个操作（即改变你维护的安装状态）。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;

typedef long long LL;

template <typename T, int SZ>
struct SegTree {
    struct Node {
        T sumv, setv;
        bool hasSet;
        void setVal(const T& val, int L, int R) {
            setv = val, hasSet = true, sumv = (R - L + 1) * setv;
        }
    }; // sumv: 最新的和; setv: 最新的 set 标记（子孙不受影响）
    Node NS[SZ];
    int N;
    void init(int _n) {
        N = _n;
    }
};
```



```

    assert((1 << ((int)ceil(log2(N)))) < SZ);
}

void pushdown(int o, int L, int R) {
    Node& nd = NS[o];
    if (!nd.hasSet) return;
    int lc = 2 * o, rc = 2 * o + 1, M = (L + R) / 2;
    NS[lc].setVal(nd.setv, L, M), NS[rc].setVal(nd.setv, M + 1, R);
    nd.hasSet = false;
}

void setV(int l, int r, const T& v) { update(1, 1, N, v, l, r); }
void update(int o, int L, int R, const T& v, int qL, int qR) {
    if (qL <= L && qR >= R) {
        NS[o].setVal(v, L, R);
        return;
    }
    int lc = 2 * o, rc = 2 * o + 1, M = (L + R) / 2;
    pushdown(o, L, R);
    if (qL <= M) update(lc, L, M, v, qL, qR);
    if (qR > M) update(rc, M + 1, R, v, qL, qR);
    NS[o].sumv = NS[lc].sumv + NS[rc].sumv;
}

T querysum(int l, int r) { return query(1, 1, N, l, r); }
T query(int o, int L, int R, int qL, int qR) {
    if (qL <= L && qR >= R) return NS[o].sumv;
    pushdown(o, L, R);
    int lc = 2 * o, rc = 2 * o + 1, M = (L + R) / 2;
    T s = 0;
    if (qL <= M) s += query(lc, L, M, qL, qR);
    if (qR > M) s += query(rc, M + 1, R, qL, qR);
    return s;
}
};

```

```

template <int SZ = 1004>
struct HLD {
    // 树链剖分
    int N, Fa[SZ], HcHead[SZ], Dep[SZ], HcTail[SZ], HSon[SZ], Usz[SZ];
    int ID[SZ], segSz;
    vector<int> G[SZ];
    void init(int _n) {
        segSz = 0;
        N = _n;
        assert(_n < SZ);
    }
    int dfs(int u, int fa) { // 返回子树体积
        int &h = HSon[u], &sz = Usz[u];
        sz = 1, Fa[u] = fa, h = 0, Dep[u] = Dep[fa] + 1;
        for (size_t i = 0; i < G[u].size(); i++) {

```

```

        int v = G[u][i];
        if (v == fa) continue;
        sz += dfs(v, u);
        if (Usz[v] > Usz[h]) h = v; // 体积最大的子树
    }
    return sz; // DFS 得到重儿子, 深度, 父结点
}

void hld(int u, int fa, int head) { // 轻重剖分
    ID[u] = ++segSz, HcHead[u] = head;
    if (Hson[u]) {
        hld(Hson[u], u, head); // 重链向下扩展
        for (size_t i = 0; i < G[u].size(); i++) {
            int v = G[u][i]; // 轻儿子新开重链
            if (v != fa && v != Hson[u]) hld(v, u, v);
        }
        return;
    }
    HcTail[head] = u;
}

void addEdge(int u, int v) { G[u].push_back(v); }
void build(int root = 1) { dfs(root, 0), hld(root, 0, root); }
};

const int NN = 1e5 + 8;
SegTree<int, NN * 3> St;
HLD<NN> H;
const int Root = 1;
int queryRootPathSum(int u) { // 查询 u 到树根路径上所有点的权值之和
    int ans = 0;
    while (true) {
        int hu = H.HcHead[u];
        ans += St.querysum(H.ID[hu], H.ID[u]);
        if (hu == Root) break;
        u = H.Fa[hu];
    }
    return ans;
}

void setRootPath(int u) { // 设置 u 到树根路径上所有点的权值为 1
    while (true) {
        int hu = H.HcHead[u];
        St.setV(H.ID[hu], H.ID[u], 1);
        if (hu == Root) break;
        u = H.Fa[hu];
    }
}

int querySubTreeSum(int u) { // 子树 u 的所有点权之和, 树上所有点在 DFS 序中是连续的
    return St.querysum(H.ID[u], H.ID[u] + H.Usz[u] - 1);
}

void clearSubTree(int u) { // 设置子树 u 的所有点权为 0

```

```

    St.setV(H.ID[u], H.ID[u] + H.Usz[u] - 1, 0);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int N, Q;
    cin >> N, H.init(N + 1), St.init(N + 1);
    for (int u = 2, p; u <= N; u++) cin >> p, H.addEdge(p + 1, u);
    H.build(Root), cin >> Q;
    string s;
    for (int i = 0, x; i < Q; i++) {
        cin >> s >> x, ++x;
        if (s[0] == 'i') { // 安装 x
            int s0 = queryRootPathSum(x);
            setRootPath(x);
            cout << queryRootPathSum(x) - s0 << endl;
        } else { // 卸载 x
            int s0 = querySubTreeSum(x);
            clearSubTree(x);
            cout << s0 << endl;
        }
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.7 节例题 39
同类题目：Black and White Tree, Codechef GERALD2
树的统计 COUNT, ZJOI 2008, 牛客 NC 20477
*/

```

例 4-32 【基于迭代的树链剖分】要有彩虹（Let there be rainbows, IPSC 2009 Problem L）

给出一棵包含 N ($N \leq 2 \times 10^6$) 个结点的树，结点编号为 $0 \sim N-1$ ，一开始所有的边都是灰色的。每次给出两个整数 x 和 y ，以及 7 种颜色（red, orange, yellow, green, blue, indigo, violet）之一的颜色 c 。将点 x 到 y 路径上每条颜色不是 c 的边涂成 c ，最终输出每种颜色分别涂了多少条边。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i,a,b) for( int i=(a); i<(int)(b); ++i)
#define _rep(i,a,b) for( int i=(a); i<=(int)(b); ++i)
typedef long long LL;
struct IntTree { // 每条重链对应的线段树，IPSC 不卡内存，如果其他 OJ，可以考虑动态开点
    struct Node {
        int color, sum[8]; // color: 区间颜色 (0 无颜色); sum: 颜色的种数
        void setc(int c, int len) {
            color = c;
```

```

        fill_n(sum, 8, 0), sum[c] = len;
    }
    Node() { setc(0, 0); }
};

int L;
vector<Node> data;
IntTree(int N) { L = 1 << (int)(ceil(log2(N + 2))), data.resize(2 * L); }
void insert(int l, int r, int clr, int o, int L, int len) {
    if (r <= L || l >= L + len) return;          //  $[L, L+len] \cap [l, r] = \emptyset$ 
    Node& d = data[o], &ld = data[2 * o], &rd = data[2 * o + 1];
    if (l <= L && L + len <= r) {                //  $[L, L+len] \in (l, r)$ 
        d.setc(clr, len);
        return;
    }
    if (d.color != 0) ld.setc(d.color, len / 2), rd.setc(d.color, len / 2);
    d.setc(0, 0);
    insert(l, r, clr, 2 * o, L, len / 2);
    insert(l, r, clr, 2 * o + 1, L + len / 2, len / 2);
    _for(i, 0, 8) d.sum[i] += ld.sum[i] + rd.sum[i];
}

int count(int l, int r, int clr, int o, int L, int len) {
    if (r <= L || l >= L + len) return 0;          //  $[L, L+len] \cap [l, r] = \emptyset$ 
    Node& d = data[o], &ld = data[2 * o], &rd = data[2 * o + 1];
    if (l <= L && L + len <= r) return d.sum[clr]; //  $[L, L+len] \in (l, r)$ 
    if (d.color != 0) ld.setc(d.color, len / 2), rd.setc(d.color, len / 2);
    return count(l, r, clr, 2 * o, L, len / 2)
        + count(l, r, clr, 2 * o + 1, L + len / 2, len / 2);
}

void insert(int l, int r, int clr) { insert(l, r, clr, 1, 0, L); }
int count(int l, int r, int clr) { return count(l, r, clr, 1, 0, L); }
};

const int NN = 1e6 + 8;
typedef vector<int> IVec;
IVec G[NN], CH[NN]; // 图的结构
int N, Fa[NN], Tin[NN], Tout[NN], Tsz[NN]; // 父结点, 时间戳, 子树大小
bool Vis[NN];
int PathId[NN], PathOffset[NN]; // 每个点所在的重链以及在其中的位置
vector<IVec> Paths; // 所有重链独立存放
vector<IntTree> ST; // 每条重链对应一棵线段树

void hld() {
    fill_n(Vis, N + 1, false), fill_n(Tsz, N + 1, 0);
    _rep(i, 0, N) CH[i].clear();
    Paths.clear();

    vector<int> walk; // 后续遍历的 DFS 序
    int time = 0;
    Vis[0] = true, Tin[0] = time, Fa[0] = 0;

```

```

stack<int> sv, se; // 当前处理的 u 以及下一个要处理的 u 的子结点 v 对应的边
sv.push(0), se.push(0);
while (!sv.empty()) { // 迭代版的 DFS
    ++time;
    int u = sv.top(); sv.pop();
    int e = se.top(); se.pop(); // 当前要处理的子树 v 的编号
    if (e == (int)G[u].size()) { // u 子树都已经处理完
        walk.push_back(u), Tout[u] = time, Tsz[u] = 1;
        for (auto v : CH[u]) Tsz[u] += Tsz[v]; // 子树 u 的体积
    } else {
        sv.push(u), se.push(e + 1);
        int v = G[u][e]; // u 的子结点 v
        if (!Vis[v]) {
            Vis[v] = true, Tin[v] = time, Fa[v] = u, CH[u].push_back(v);
            sv.push(v), se.push(0);
        }
    }
}

fill_n(Vis, N + 1, false);
Vis[0] = true; // u->pa[u]处理过了?
for (auto w : walk) {
    if (Vis[w]) continue;
    IVec p(w);
    while (true) {
        bool heavy = (2 * Tsz[w] >= Tsz[Fa[w]]);
        Vis[w] = true, w = Fa[w], p.push_back(w);
        if (!heavy || Vis[w]) break;
    }
    Paths.push_back(p);
}

PathId[0] = -1; // root 不在任何链上
_for(i, 0, Paths.size()) _for(j, 0, Paths[i].size() - 1) {
    PathId[Paths[i][j]] = i;
    PathOffset[Paths[i][j]] = j;
}
ST.clear();
for (const auto& p : Paths) ST.emplace_back(p.size() - 1);
}

inline bool is_ancestor(int x, int y) { // x 是 y 的祖先?
    return (Tin[y] >= Tin[x] && Tout[y] <= Tout[x]);
}

// 统计[x-y]路径上过去不是颜色 c 而这次被涂成颜色 c 的边数
int query(int x, int y, int c) {
    if (x == y) return 0;

```

```

    if (is_ancestor(x, y)) return query(y, x, c);
    int pi = PathId[x], l = PathOffset[x], r = Paths[pi].size() - 1;
    const auto& pt = Paths[pi];
    if (is_ancestor(pt[r], y)) {
        while (r - l > 1) { // 确保 r 在 LCA(x, y) 下方
            int m = (r + l) / 2;
            if (is_ancestor(pt[m], y)) r = m; else l = m;
        }
        l = PathOffset[x];
    }
    int ans = r - l - ST[pi].count(l, r, c); // 以前有多少其他颜色, 会被涂成 c
    ST[pi].insert(l, r, c);
    return ans + query(pt[r], y, c); // 加上 LCA(x, y)-y 路径上的
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    string color[] = {"", "red", "orange", "yellow", "green", "blue", "indigo",
"violet"};
    map<string, int> CI;
    _rep(i, 1, 7) CI[color[i]] = i;
    int T, Q; cin >> T;
    while (T--) {
        cin >> N;
        _rep(i, 0, N) G[i].clear();
        for (int i = 0, x, y; i < N - 1; ++i)
            cin >> x >> y, G[x].push_back(y), G[y].push_back(x);
        hld();
        cin >> Q;
        vector<LL> ans(8, 0);
        string c;
        for (int i = 0, x, y; i < Q; i++) {
            cin >> x >> y >> c;
            ans[CI[c]] += query(x, y, CI[c]);
        }
        _rep(i, 1, 7) cout << color[i] << " " << ans[i] << endl;
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.7 节例题 40
同类题目: Heavy Snowfall, IPSC 2016
*/

```

本节例题列表